

SYSTEM DESIGN

FUNCTIONAL REQUIREMENTS

1. The user can upload courses and other documents (PDF, DOCX, PPTX only) which will be vectorized and stored in the vector database on the backend.
2. The user can see the current processing status of an uploaded document (Extracting, Vectorized, Failed).
3. The user can search through uploaded materials.
4. The user can filter the uploaded materials.
5. The user can delete a document and by doing so, the raw version, the embedded chunks and any deck/flashcards related to it will be deleted.
6. The user can create a deck of flashcards (15 / 25 / 40 cards or less, depending on the available knowledge of the Agent) based on an uploaded topic.
7. The user can specify the target difficulty when generating a flashcard deck.
8. The user can rename an auto-generated deck for better personal organization.
9. The user can delete an existing flashcard deck without deleting the source document.
10. The user can search through the available decks.
11. The user can filter the available decks.
12. The user can perform basic filtering (All / Due) on the available decks.
13. The user can start answering flashcards, starting from the beginning or from the last unanswered flashcard if the deck is already started.
14. The user can stop at any time and the deck will be continued from where he left.
15. Each flashcard has the main question on the front side and it can be clicked to reveal the answer.
16. The user must give a short feedback for each question, how well he knew the answer.
17. The user can rewatch the answered flashcards, but he can not answer them again.
18. The user can view the last 3 answered questions (read-only).
19. The user can set a certain daily target (number of reviewed flashcards) and see the progress bar on different pages.
20. The user can update their display name.
21. The user can select their profile avatar from a set of 6 predefined options.
22. The user can view their current consecutive daily streak of answered flashcards, as well as the all-time longest streak.
23. The user can ask the AI Tutor certain questions about the uploaded materials.
24. The AI Tutor's responses are limited to its knowledge of the subject and any out of scope question will be politely unanswered.
25. The user's conversations with the AI Tutor are automatically saved as persistent chat sessions in the database.
26. A unique chat session title is automatically generated by the system based on the user's initial question.
27. The AI Tutor streams its responses to the user in real-time as they are being generated.
28. The user can view a history of all their past chat sessions, ordered by the most recently created.
29. The user can open any past chat session to view the complete message history of that specific conversation.

30. The user can continue an existing conversation, and the AI Tutor will maintain the context of the previous messages in that session.
31. The user can permanently delete a chat session at any time, which automatically removes all of its associated messages.

NON-FUNCTIONAL REQUIREMENTS

1. All document parsing, embedding, and AI inference must execute entirely locally, ensuring no user data is ever sent to external cloud APIs.
2. The system must restrict document uploads to a maximum size (20MB) to prevent memory crashes during local text extraction.
3. The system must provide immediate visual feedback (such as loading spinners, skeleton loaders or pop-ups) during asynchronous tasks like document uploading, vectorization, and flashcard generation.
4. The AI Tutor's responses must be streamed to the UI token-by-token to reduce perceived latency and mimic natural chat interfaces.
5. The system must fail gracefully if local backend services (ChromaDB or Ollama) are offline, displaying a friendly disconnection banner instead of crashing the app.
6. The system must safely abort the parsing process if it encounters a corrupted document, marking the file with an error status without halting the rest of the application.
7. The application interface must be fully responsive, scaling seamlessly to provide a usable experience across mobile, tablet, and desktop viewports.

ENTITIES

1. DOCUMENT ENTITY → LIBRARY CARD
 - a. id (UUID) - also used in vector database
 - b. filename
 - c. file type
 - d. title
 - e. upload date
 - f. status
 - g. number of pages
 - h. category
2. DECK ENTITY → DECK CARD
 - a. id
 - b. document id
 - c. title
 - d. category
 - e. difficulty
 - f. status
 - g. created at date
 - h. last accessed date
 - i. number of cards
 - j. last unanswered card index
3. FLASHCARD ENTITY → FLASHCARD
 - a. id
 - b. deck id
 - c. index / position in deck
 - d. question

- e. answer
 - f. difficulty
 - g. feedback answer
4. ANSWER LOG ENTITY → ANSWERED QUESTION CARD
- a. id
 - b. flashcard id
 - c. deck id
 - d. answered date
5. USER ENTITY → USER
- a. id - for multiple users in the future
 - b. created at date
 - c. timezone
 - d. last active date
 - e. username
 - f. daily target
 - g. current answer streak
 - h. longest answer streak
 - i. avatar id
6. CHAT MESSAGE ENTITY → CHAT MESSAGE
- a. id
 - b. session id
 - c. created at date
 - d. role
 - e. content
7. CHAT SESSION ENTITY → CHAT SESSION
- a. id
 - b. user id
 - c. created at date
 - d. title

API

1. DOCUMENTS

POST /api/documents → Upload document, parse and vectorize it

GET /api/documents → Retrieve all uploaded documents

GET /api/documents/?id → Get specific document details

DELETE /api/documents/?id → Delete the document and also the embeddings, decks and flashcards related to it

2. DECKS & FLASHCARDS

POST /api/deck → Generate a new deck based on a document using a LLM

GET /api/decks → List available decks

GET /api/decks/?id → Retrieve a specific deck

PATCH /api/decks/?id → Rename deck or change category

DELETE /api/decks/?id → Delete a deck without affecting the source document

GET /api/flashcards/?id → List all flashcards generated from a deck

PATCH /api/flashcards/?id → Submit user feedback for a flashcard

GET /api/flashcards/recent → Fetch the last 3 answered flashcards

3. USER ACTIONS

POST /api/user → Create a new user with specific timezone and name

GET /api/user → Get user information such as name, daily target

PATCH /api/user → Update user information

GET /api/user/daily → Get user daily progress: answered cards & target

4. CHAT

POST /api/chat → Ask the AI Tutor a question based on local documents

GET /api/chat/users/?user_id/sessions → Get chat sessions for a specific user

GET /api/chat/sessions/?id → Get details about a session

GET /api/chat/sessions/?session_id/messages → Get all messages from a session

DELETE /api/chat/sessions/?id → Delete a chat session

HIGH LEVEL DESIGN

A link to an Excalidraw diagram can be found here: [Excalidraw Link](#)



